Neural networks fundamentals

Course: LING 285: Human Language and Technology
Title of Lecture: Lecture 10: Neural Networks 1.5
Date: 9/24/2026
Professor: Khalil Iskarous

Section 1: The Job of a Neural Network

- Picture a task where you are handed a huge number of photographs of faces, each photograph made up of 48 million pixels, and your job is to say which one of 1000 possible people is shown in each picture. A second, parallel task hands you speech waves instead of photographs: for each wave you are asked to say which of 25 possible sounds that wave corresponds to. Both tasks have the same underlying shape, since in each case you are given a large block of numbers and asked to sort it into one of a fixed set of categories.
- The tool built to solve problems shaped like this is called a neural network, abbreviated NN. The reason for that particular name is deliberately set aside for now, because the concept is going to be built up entirely from what the network does and how it behaves, described here as learning what it is by watching how it walks and how it swims rather than by being told its label first.
- The job of a neural network is to take a large bunch of numbers, whether those numbers are the 48 million pixel values of one image or the numeric samples of a sound wave, and to return a probability distribution over the possible categories. For the speech task, that returned probability distribution has 25 entries, one for each of the 25 possible sounds.
- A simplified version of the speech task keeps only 5 vowels instead of 25 sounds, so the network's output becomes a five-number probability distribution. An example output pdf is given as $[.2, .2, .4, .2, 0]$, where the five vowel categories are agreed to be listed in the order read, rad, rude, Rud, rod.
    - A defining property of any such output is that its numbers, the p's, must sum to 1, which is the same as saying they sum to 100 percent.
    - Reading the example pdf against the agreed vowel order says the following: there is a 20 percent chance the sound is the vowel in read, a 20 percent chance it is the vowel in rad, a 40 percent chance it is the vowel in rude, a 20 percent chance it is the vowel in Rud, and a 0 percent chance it is the vowel in rod.

Section 2: Weights, Forward Propagation, and Learning by Trial and Error

- The input to a neural network is described generally as a bunch of numbers, a signal, or a vector, with an example given as $[4, 1, -3, 2, 1, .4, 1, 2, -3, \ldots]$, and in practice such an input vector usually contains thousands of numbers. In the speech case, this vector is made up of the pressure numbers that describe a speech wave over time.
- The output of the network is a decision pdf, meaning that given the input signal, the network reports the probability of each possible object it could be, which in the vowel example means the probability of each of the vowels in read, rad, rude, Rud, and rod.
- The mechanism that lets a network arrive at this decision is that it holds knowledge inside a large collection of weights, referred to as w's. The input signal vector is combined with these weights, and through many, many calculations, a decision about which object was detected eventually emerges. The scale involved is enormous, with several trillion weights and billions of calculations taking part in producing a single decision.
- These weights are what allow the network to make decisions, which raises the question of how the network ever comes to know the correct values for them. The answer given here is summarized as fake it till you make it, applied through a procedure called stochastic gradient descent, or SGD, for neural networks.
    - The procedure starts by picking random values for all of the weights, meaning the network begins with no real knowledge at all.
    - The network is then given one training signal, with the example used here being a signal corresponding to the vowel in Rud.

    - This step is called forward propagation: the network uses its current, still essentially random and unhelpful weights and produces an output pdf that does not reflect reality, illustrated here as $[.4, .1, .2, .2, .1]$.
    - The network is then told that this answer is wrong, and that it should have produced $[0, 0, 0, 1, 0]$, the distribution that places all of the probability on Rud. Comparing its actual output to this correct target lets the network register that it has made a large error, also called a loss.
    - The network then adjusts its weights in a way that is meant to reduce that loss.
    - This entire cycle, of receiving a signal, producing a pdf, comparing it against the correct answer, and adjusting the weights, is repeated over and over again, billions of times, using billions of different training signals.
- This raises a further question that is posed directly without being answered yet: what calculations are actually being performed on the signal itself as it passes through the network.

Section 3: Two Huge Questions

- The material identifies two huge questions that any account of a neural network has to answer. The first is a learning question: how does a neural network learn its weights, specifically. The second is an inference question: how does a neural network combine its weights with its inputs, specifically.
- The plan stated is to answer the second question, inference, before the first question, learning, and the stated reason is that the inference question is actually part of the learning question, since learning depends on first understanding how weights and inputs are combined.

Section 4: Neurons, Activity, and Synaptic Weights

- The starting building block is that one neuron computes a weighted sum. This is connected to Santiago Ramón y Cajal's foundational claim that the brain is made of neurons, and the specific mathematical operation involved is named in several equivalent ways: an inner product, a scalar product, a dot product, or, in continuous form, the integral $\int f \cdot g$.
- This idea is grounded in an actual historical image, Cajal's drawing of a retinal circuit, taken from his 1917 memoir Recuerdos de mi vida, volume II, figure 40. In the image as used here, the peripheral half of the original figure has been cropped and rotated so that the main direction of signal flow runs left to right, and the circles drawn onto the image mark the visible neuronal cell bodies, or somata.
- The image is built up in a specific sequence to turn Cajal's anatomical drawing into an arithmetic statement.
    - Circles first appear directly over the neuronal somata in the drawing.
    - Positive and negative activation numbers then appear over those circled somata.
    - Arrows are drawn in to identify a specific, selected set of synaptic contacts among the many present in the original anatomy.
    - The signals are shown traveling rightward through those selected contacts, matching the rightward orientation the image was rotated into.
    - The arrows that mark those contacts are then relabeled as synaptic weights, using the variable w.
    - Those w labels are then replaced by their actual signed numerical weight values.
    - The picture is simplified down to only three inputs feeding into one receiving neuron, discarding the rest of the anatomical detail.
    - The three surviving connections, or edges, carry the specific weights -2, 3, and 1.
    - The three input activations feeding into those edges are -1, 1, and 3.
    - The receiving neuron is shown computing the inner product of these weights and activations, which is the weighted sum described at the start of this section.
- That inner product is worked out explicitly as

$$(-1 \times -2) + (1 \times 3) + (3 \times 1) = 8$$

- This three-input example is described as an artificial circuit built for teaching purposes: the weights are $[-2, 3, 1]$, the activations are $[-1, 1, 3]$, the weighted sum is 8, and no bias term is included in the calculation.
- It is also specified that the signed numerical activations used here are artificial-neuron quantities constructed for the demonstration, not a claim about actual measured negative firing rates in real neurons, and that the gold-colored packets and the selected contacts drawn onto the image are schematic teaching overlays rather than a full anatomical reconstruction or a claim about measured neural activity.

Section 5: Brain Chemistry and Electronics Turned Into Arithmetic

- Three landmark publications are presented in chronological order to trace how ideas about the brain's chemistry and electrical activity were progressively turned into arithmetic that machines could carry out.
- The first, dated 1943, is the paper "A Logical Calculus of the Ideas Immanent in Nervous Activity" by Warren S. McCulloch and Walter Pitts, whose listed affiliations are the University of Illinois College of Medicine, the Department of Psychiatry at the Illinois Neuropsychiatric Institute, and the University of Chicago in Chicago, U.S.A.
- The second, dated 1962, is Frank Rosenblatt's book Principles of Neurodynamics, with the subtitle Perceptrons and the Theory of Brain Mechanisms, published by Spartan Books.
- The third, dated 1986, is Parallel Distributed Processing: Explorations in the Microstructures of Cognition, Volume 1: Foundations, by David E. Rumelhart, James L. McClelland, and the PDP Research Group.
- Taken together, these three works span 1943 to 1986 and represent the line of work described as translating brain chemistry and electrical activity into arithmetic.

Section 6: Beliefs Not on the Exam

- Two contrasting beliefs about the relationship between modern AI and the brain are laid out, explicitly marked as not being exam material.
- The first belief is attributed to most engineers, neuroscientists, and people generally: that modern AI has nothing to do with how humans think, perceive, act, or decide, because the brain is full of chemistry, electrical activity, and abstract processes, none of which seem to have anything to do with large language models, which are made only of numbers and arithmetic.
- The second belief is attributed to a few people, including the professor: that modern AI is the product of roughly 100 years of evolutionary tinkering aimed at figuring out how arithmetic can emulate the brain's chemical and electrical activity, and that these attempts appear to have succeeded.

[CONTINUE]

Section 7: A Multilayer Perceptron

- The network shown is a multilayer perceptron with five layers of neurons in sequence: an Input layer, Hidden 1, Hidden 2, Hidden 3, and an Output layer. The neuron counts in these layers are 5, 3, 4, 3, and 7 respectively, so the architecture is described as 5-3-4-3-7, and every neuron in each layer is connected to every neuron in the next layer, giving 60 weighted connections in total across the four adjacent-layer connection sets.
- For this numerical demonstration, no bias terms are used anywhere in the network, and the hidden layers use linear activations, meaning each hidden neuron's activation is simply the weighted sum of its inputs with no additional nonlinear squashing function applied. This is explicitly flagged as a simplification: a normal multilayer perceptron introduces nonlinear hidden activation functions, and that extra step has been deliberately left out here so the arithmetic stays transparent.
- The input activations for this worked example are $[2, 1, 3, 4, 2]$, one number for each of the five input neurons.
- One particular fan of connections is highlighted: all five input neurons feed into the middle neuron of Hidden 1, which is Hidden 1's second neuron, and the weights on that

specific fan are $[2, -1, 1, 3, -2]$.

- The full set of weights connecting each pair of adjacent layers is given as destination-by-source matrices, meaning each row corresponds to one receiving neuron and lists the weights coming from every neuron in the layer before it.
  - The weights from Input to Hidden 1 are
$$W_1 = \begin{bmatrix} 0.5 & 1 & 1 & -0.5 & 0.5 \\ 2 & -1 & 1 & 3 & -2 \\ -1 & 0.5 & 0.5 & -1 & 0.5 \end{bmatrix}$$
  - The weights from Hidden 1 to Hidden 2 are
$$W_2 = \begin{bmatrix} 1 & 0.25 & 0.5 \\ 0.5 & -0.5 & -1 \\ -1 & 0.75 & 0.5 \\ 0.5 & 0.25 & 1.5 \end{bmatrix}$$
  - The weights from Hidden 2 to Hidden 3 are
$$W_3 = \begin{bmatrix} 0.5 & 1 & 0.5 & -0.5 \\ -0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.1 & -0.5 \end{bmatrix}$$
  - The weights from Hidden 3 to Output are
$$W_4 = \begin{bmatrix} 0.0009145128098591027 & -0.00030483760328636755 & 0.0006096752065727351 \\ 0.14944605150127596 & -0.04981535050042533 & 0.09963070100085065 \\ 0.2979775901926928 & -0.09932586339756426 & 0.19865172679512852 \\ 0.3848629705015851 & -0.1282876568338617 & 0.2565753136677234 \\ 0.2363314318101683 & -0.07877714393672276 & 0.15755428787344553 \\ 0.18851495652855194 & -0.06283831884285064 & 0.12567663768570128 \\ 0.10162957621965951 & -0.03387652540655317 & 0.06775305081310634 \end{bmatrix}$$
- Passing the input vector through the network layer by layer under these linear, bias-free rules produces the following sequence of activation vectors, moving from the input all the way to Hidden 3:
$$[2, 1, 3, 4, 2] \quad \rightarrow_{\text{layer}} \quad [4, 14, -3] \quad \rightarrow_{\text{layer}} \quad [6, -2, 5, 1] \quad \rightarrow_{\text{layer}} \quad [3, -1, 2]$$
  - Since the middle neuron of Hidden 1 is the second entry of the Hidden 1 vector, its activation of 14 is exactly the value produced by taking the weighted sum of the input vector $[2, 1, 3, 4, 2]$ with that neuron's own weight row $[2, -1, 1, 3, -2]$, consistent with the highlighted fan described above.
- At the output layer, the network's raw scores, called logits, are related to the seven displayed output probabilities $p = [.05, .10, .20, .30, .15, .12, .08]$ by the relationship
$$z = \log(p) + 3$$
  - Multiplying the Hidden 3 activation vector by W_4 produces exactly this logit vector z.
  - Applying the softmax function to these logits is what produces the displayed probabilities. Softmax works by taking a vector of raw, unconstrained scores, exponentiating each one, and then dividing each exponentiated score by the sum of all of the exponentiated scores, which guarantees that every resulting value is positive and that the whole set of resulting values sums to exactly 1.
  - The seven probabilities produced this way are $0.05, 0.10, 0.20, 0.30, 0.15, 0.12$, and 0.08, which sum exactly to 1.00.
  - All of the numbers used in this demonstration, the input activations, the four weight matrices, and the resulting probabilities, are stated to form one internally consistent illustrative example rather than being drawn from an actual trained network.

Section 8: Pop Quiz on Hidden 1 Neuron 2

- The same 5-3-4-3-7 network, with the same four weight matrices W_1, W_2, W_3, and W_4 described above, is reused for a comprehension check, this time with a new set of input activations, $[-2, 1, 3, -4, 2]$, in place of the earlier demonstration's $[2, 1, 3, 4, 2]$.
- The question posed is what the activation on Hidden 1's second neuron is under this new input, using the same linear, bias-free rule established earlier, where a hidden neuron's activation is the weighted sum of the previous layer's activations with that neuron's own

row of incoming weights, which for this neuron is the same row $[2, -1, 1, 3, -2]$ used in the demonstration in Section 7.
- The output layer's circles are left unlabeled for this version, since the question only asks for this one intermediate hidden-layer activation rather than the full output probability distribution.

Section 9: Learning Through FITYMI and Backpropagation

- Learning is approached with the same fake-it-till-you-make-it, or FITYMI, attitude introduced earlier: the network's weights are picked randomly, and a training input/output pair is picked randomly to learn from.
- The example pair given is an input $x = [-2, 1, 3, -4, 2]$ paired with a target output pdf of $[0, 0, 0, 0, 0, 1, 0]$, meaning the correct answer places all of the probability on the sixth of the seven output categories.
- Whatever pdf the network actually produces for this input using its current, randomly chosen weights is treated as wrong, since the actual, correct pdf at the output should be $[0, 0, 0, 0, 0, 1, 0]$.
- The weights are then changed so that the network's output pdf moves closer to this true target distribution. This weight-adjustment procedure is stochastic gradient descent, the same SGD introduced earlier for neural networks, and in this specific context it is now given the name backpropagation.
- This entire process, drawing a training pair, comparing the network's output to the correct pdf, and nudging the weights to reduce the difference, is repeated over and over again across billions of input/output pairs, and this repeated, large-scale weight adjustment is identified as exactly what is carried out at data centers when large neural networks are trained.

Section 10: Python SGD for Estimating an Average

- A simpler numerical example is introduced to illustrate stochastic gradient descent outside of the neural network setting: the goal is to estimate a single value, called w, meant to be the average of the list of numbers $[73, 90, 98, 46, 400, 81, 82, 78, 62, 91, 61, 84, 75, 84, 81, 75]$.
- One of the numbers in this list, 400, is present by mistake.
- Computing the arithmetic average of the list exactly as written, mistake included, gives a value of about 100.
- In reality, since the datapoint 400 is understood to be an error, the true average of the list, calculated without that mistaken value, is about 80.
- For simplicity, the list as written is treated as though it is already in the order in which its values will be encountered one at a time, at random, during the stochastic gradient descent procedure used to estimate w.